

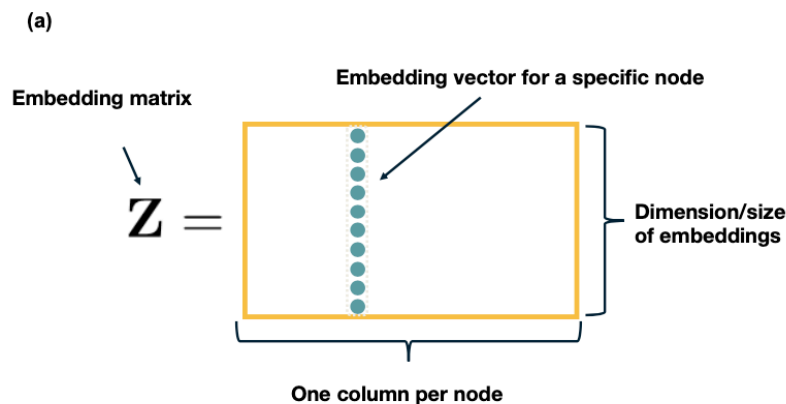
L14: Shallow Encodings and Similarity Metrics

In this page we discuss some simple approaches to compute node embeddings. They are referred to as "shallow" because the computation of the embeddings does not require to train a deep neural network.

If we have n nodes and the dimensionality of the embeddings is d , the node embeddings can be "encoded" in a $d \times n$ matrix $\mathbf{Z}$ (see figure below). If $\mathbf{v}$ is an $n \times 1$ "one-hot" vector (all elements are zero except the element that represents node v , which is one), then the embedding of node v is given by the following matrix multiplication:

$$\text{ENC}(v) = \mathbf{Z} \mathbf{v}$$

which simply extracts the i 'th vector of matrix $\mathbf{Z}$.



Images from Jure Leskovec, "Representation Learning on Networks" tutorial, WWW 2018

Let us now see how to compute the matrix $\mathbf{Z}$ based on different notions of "node similarity".

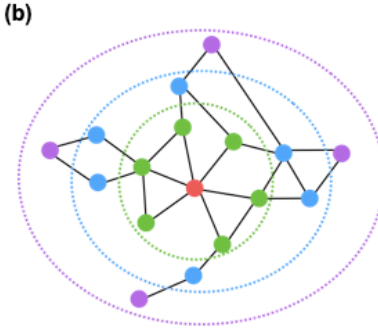
The simplest approach is to define that *two nodes are similar only if they are directly connected*. Further, if the network is weighted, the similarity of two nodes can be equal to the weight of their connecting edge. Specifically, if $\mathbf{A}$ is the (possibly weighted) $n \times n$ adjacency matrix of the network, we can compute the matrix $\mathbf{Z}$ so that the dot-product of the embeddings $\mathbf{z}_u$ and $\mathbf{z}_v$ of any two nodes u and v is equal to the element $\mathbf{A}_{u,v}$ of the adjacency matrix. Recall that the dot-product of two vectors with unit magnitude is equal to the cosine of the angle between them -- and so it quantifies the distance between the two vectors.

One way to compute the matrix $\mathbf{Z}$ then, is to minimize the following "loss function" through any numerical optimization method (such as Stochastic Gradient Descent):

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^T \mathbf{z}_v - \mathbf{A}_{u,v}\|$$

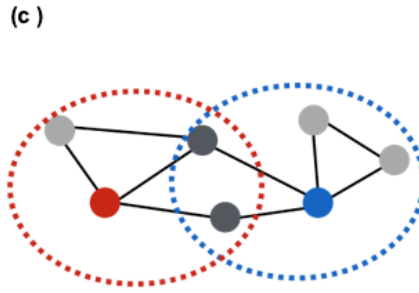
If the previous loss cannot be minimized exactly to zero, the node embeddings would only approximate the previous relation with the adjacency matrix. An obvious drawback of this approach is that two nodes are considered "not similar" if they are not directly connected, even if they have many common network neighbors.

A more general approach is to consider that a node v is "similar", not only to its direct neighbors, but *also to any node that is at most k -hops away from v* (for a small value of k , say 2 or 3). For example, in the following figure, the red node would be considered "similar" to only the green nodes if $k=1$, also the blue nodes if $k=2$, and all shown nodes if $k=3$. Then, we can compute the embedding matrix $\mathbf{Z}$ using the previous loss function, but replacing the adjacency matrix $\mathbf{A}$ with a k -hop adjacency matrix $\mathbf{A}_k$ in which two nodes are considered neighbors even if they are at most k -hops away from each other. Another approach is to use the first k powers of the adjacency matrix $\mathbf{A}^k$. Recall from Lesson-2 that this matrix gives the number of k -hop paths between any two nodes (for unweighted graphs).



Images from Jure Leskovec, "Representation Learning on Networks" tutorial, WWW 2018

Another approach is to consider the overlap between the network neighborhood of two nodes. For instance, in the following network the blue and red nodes are not directly connected but they have two common neighbors (out of 3 neighbors for the red node, and out of 4 neighbors for the blue node).



Images from Jure Leskovec, "Representation Learning on Networks" tutorial, WWW 2018

Recall that we had defined such a node similarity metric in Lesson-7. Specifically, the similarity $S_{i,j}$ between any pair of nodes i and j can be defined as:

$$S_{i,j} = \frac{N_{i,j} + A_{i,j}}{\min\{k_i, k_j\}}$$

where $N_{i,j}$ is the number of common neighbors of i and j , $A_{i,j}$ is the adjacency matrix element for the two nodes (1 if they are connected, 0 otherwise), and k_i is the degree of node i . Note that $S_{i,j} = 1$ if the two nodes are connected with each other and every neighbor of the lower-degree node is also a neighbor of the other node. On the other hand, $S_{i,j} = 0$ if the two nodes are not connected to each other and they do not have any common neighbor. There are several such node similarity metrics in the literature with minor differences between them.

Given such a similarity metric, the node embeddings can be then computed numerically based on the following loss function:

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^T \mathbf{z}_v - S_{u,v}\|$$

All three approaches that we have seen in this page have some common characteristics. First, they are rather computationally expensive because the loss function considers every possible pair of nodes, even if the similarity between the two nodes is quite weak. This drawback can be addressed using appropriate regularization terms in the previous loss functions that ignore node pairs with very weak similarity.

A second drawback is that these methods require a separate embedding vector for each node, increasing the number of parameters of the resulting machine learning model proportionally to the network size. Ideally, we would prefer a node embedding scheme in which the number of model parameters is either constant or it scales sub-linearly with the network size. We will see such schemes later in this lesson.

Food For Thought

- 1) The previous loss functions rely on the dot-product of embedding vectors. How would you modify these loss functions so that they rely instead on the L2-distance of embedding vectors?
- 2) Which of the previous methods would you prefer if the network is weighted and directed?